

平衡树 Splay

平衡树Treap是根据随机给一个节点分配优先级，并且通过左旋和右旋的方法让一个二叉搜索树尽量平衡。因为具有随机性，所以时间复杂度不会很高，但是也不会很低，而且具有不稳定性。

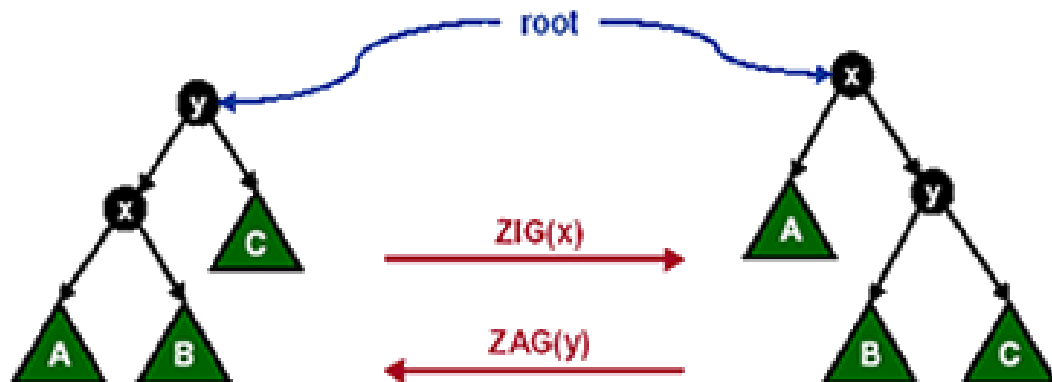
让一棵二叉搜索树平衡的方法有很多种，为了使整个查找时间更短，我们可以把查找频率高的数搬移到离根最近的地方。Splay就是在每次查询的时候，通过一系列的旋转，把某个节点搬到根上去。

Splay相对于其他二叉搜索树，他的效率比较高，冗余比较少，所需要分配的空间比较小，并且可以支持区间操作。

伸展操作

通过前面我们对平衡树Treap的学习，我们知道对二叉搜索树任意某个点进行左旋或右旋操作，不会影响二叉搜索树的性质。同样的，Splay的各项操作也是基于旋转，只不过和Treap有一点区别。

第一种：如果当前节点x的父亲节点y是根结点(Zig/Zag)。

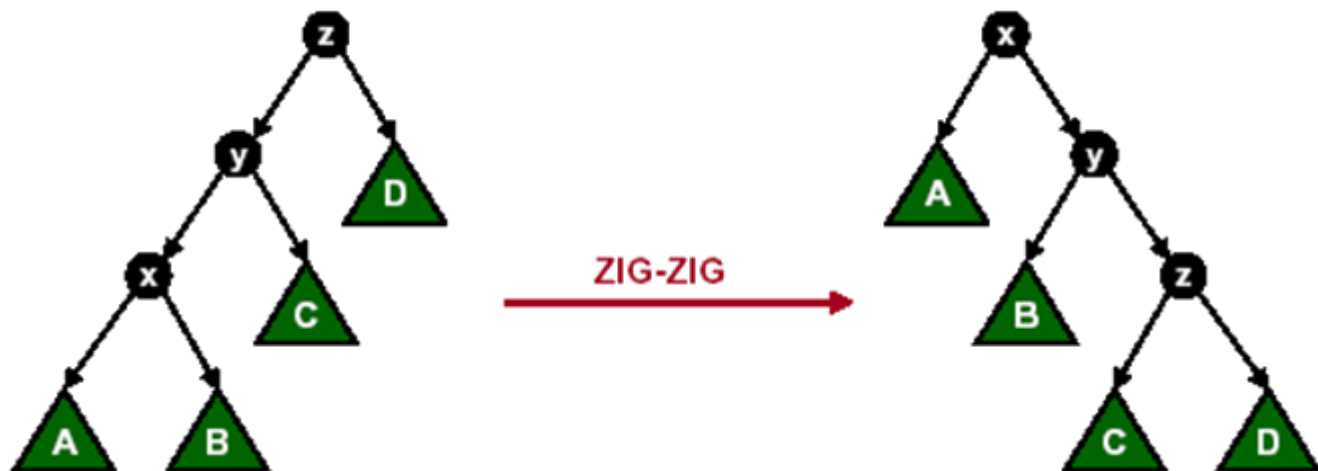


和Treap的左旋
和右旋一样

将节点x通过旋转提升一层

伸展操作

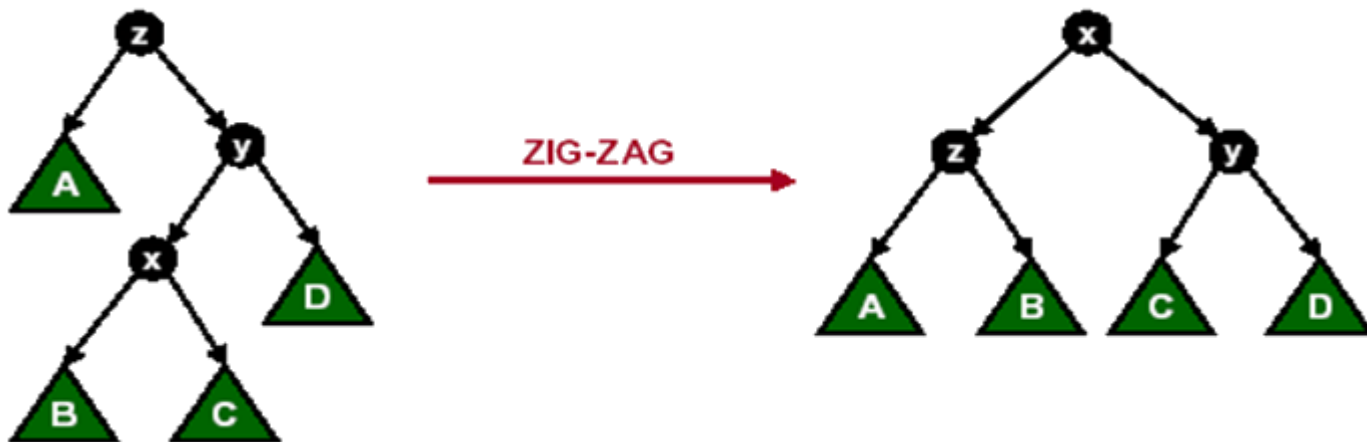
第二种：如果当前节点x的父亲节点y不是根结点，并且x,y都是其父亲节点的左孩子或者都是右孩子（Zig-Zig/Zag-Zag）。



执行步骤：先对当前节点x的父亲节点y执行Zig操作，再对当前节点x执行Zig操作（不是对x执行两次Zig操作）。把该结点x高度提升两层。

伸展操作

第三种：如果当前节点x的父亲节点y不是根结点，并且x,y 其中一个为左孩子而另一个是右孩子（Zig-Zag/Zag-Zig）。



执行步骤：对当前节点x先执行一次Zig操作，再执行一次Zag操作。把该结点x高度提升两层。

Splay博客：

<https://blog.csdn.net/octopusflying/article/details/51888826>

伸展操作

```
void rotate(int x)
{
    int y=tree[x].fa;
    int z=tree[y].fa;
    int k=tree[y].ch[1]==x; //k表示x是y的左儿子还是右儿子
    tree[z].ch[tree[z].ch[1]==y]=x; //和上面一样
    tree[x].fa=z;
    tree[y].ch[k]=tree[x].ch[k^1]; //一定都要同时更新父亲和儿子
    tree[tree[x].ch[k^1]].fa=y;
    tree[x].ch[k^1]=y;
    tree[y].fa=x;
    update(y); //y是x的儿子，一定要自下而上更新size
    update(x);
}
```

插入和查找

Splay树的插入操作和普通二叉搜索树的插入方法相同。不同的地方在于，插入操作执行完之后，对插入结点执行Splay操作，使插入节点的**父亲节点**成为该splay树的根。这样做的目的是平摊整个算法中不同操作的执行时间，使得整个算法的平摊效率趋近于 $O(\log n)$

Splay树的查找操作和普通二叉搜索树的插入方法相同。不同的地方在于，查询操作执行完之后，对查询结点执行Splay操作，使其成为该splay树的根。

插入

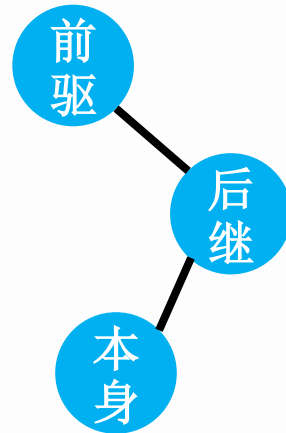
```
void add(int x)
{
    int now=root,fa=0;//now表示当前位置, fa用来记录父亲节点
    while(now&&tree[now].val!=x)
    {
        fa=now;|
        now=tree[now].ch[x>tree[now].val];//大于, 右子树, 小于, 左子树
    }
    if(now)//重复元素+1
    tree[now].cnt++;
    else//如果以前不存在就新建一个结点
    {
        now=++k;
        if(fa)
        tree[fa].ch[x>tree[fa].val]=now;
        tree[k].fa=fa;
        tree[k].size=tree[k].cnt=1;
        tree[k].val=x;
    }
    splay(now,0);//一定要有这个操作, 因为splay的时候更新size
}
```


查找

```
void search(int x)
{
    int now=root;
    if(!now)
        return ;
    while(tree[now].ch[x>tree[now].val]&& x!=tree[now].val)
        now=tree[now].ch[x>tree[now].val];
    splay(now,0); // 顺便就旋转到根结点
}
```

删除操作

Splay树的删除操作和Treap树的删除操作不一样，Treap树的删除操作是把要删除结点旋转到叶子结点或者只有一个子树的节点，再执行删除操作。而Splay树的操作都是把当前节点移动到根结点再做操作。我们可以先找到当前节点的前驱节点，把该结点移动到根结点。再找到当前节点的后继节点，把该结点移动到一个根结点的下方（右孩子）。我们需要删除的节点在前驱节点和后继节点之间。即根结点的右孩子的左孩子，直接删除就可以了。

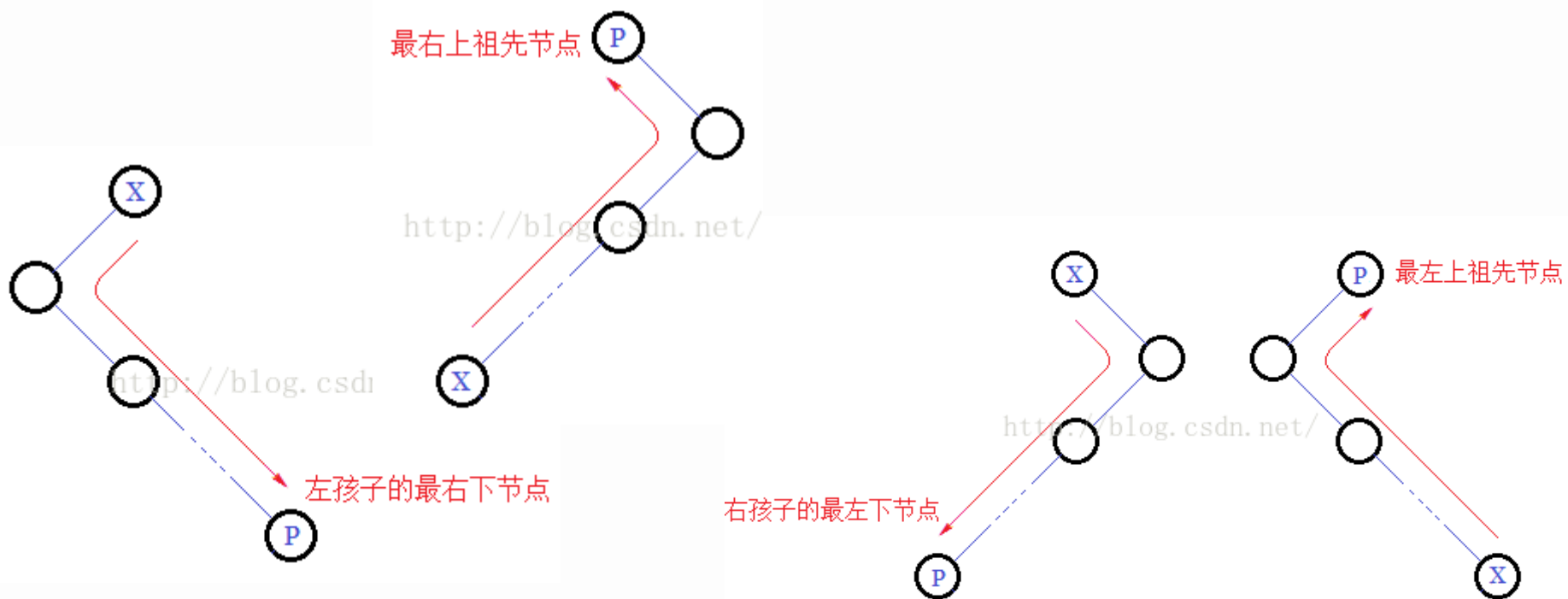


删除

```
void del(int x)//这种写法效率特别低!!!
{
    int t1=pre(x);//找到x的前驱
    int t2=net(x);//找到x的后继, 不能直接x-1和x+1, 因为没得这个数
    //也可以直接把x旋根, 然后直接删除, 再把两棵子树合并
    splay(t1,0);
    splay(t2,t1);
    int tmp=tree[tree[root].ch[1]].ch[0];
    if(tree[tmp].cnt>1)//如果不止一个就减少cnt数量
    {
        tree[tmp].cnt--;
        splay(tmp,0);
    }
    else//如果只有一个就删除结点, 即根结点的右子树的左儿子为空
    tree[tree[root].ch[1]].ch[0]=0;
}
```

前驱和后继

Splay树求前驱后继的方法和Treap树的方法是相同的。前驱就是该结点的左子树的最右节点。后继是该结点的右子树的最左节点。但是值得注意的是。如果该结点没有左孩子(右孩子)。前驱和后继应该在其祖先节点上。



前驱和后继

```
int pre(int x)//求前驱
{
    search(x);//把x旋根
    int now=tree[root].ch[0];//前驱一定在x的左子树上
    while(tree[now].ch[1])//左子树上最右边那一个
        now=tree[now].ch[1];
    return now;
}
int net(int x)//求后继, 和求前驱刚好相反
{
    search(x);
    int now=tree[root].ch[1];
    while(tree[now].ch[0])
        now=tree[now].ch[0];
    return now;
}
```

区间操作

Splay树相对于普通平衡树最大的优势在于他可以支持区间操作。

一：删除序列中在区间 $[a, b]$ 之间的数。

Splay树的点删除方法是把该点的前驱提根，该点的后继提为根的右孩子，从而删除根的右孩子的右孩子。同样的，我们可以把区间 $[a, b]$ 看做一个整体，求出区间 $[a, b]$ 的前驱和后继，然后按照前面的方法，根的右孩子的左子树就是我们需要删除的区间，直接删除即可。即寻找 a 的前驱和 b 的后继。

区间操作

二：求已知序列中区间 $[a, b]$ 中第 k 大的值

求解方法和前面一样，求出 a 的前驱提根， b 的后继提为根的右孩子。那么跟的右孩子的左子树就表示区间 $[a, b]$ ，在该子树中求出第 k 大就可以了。

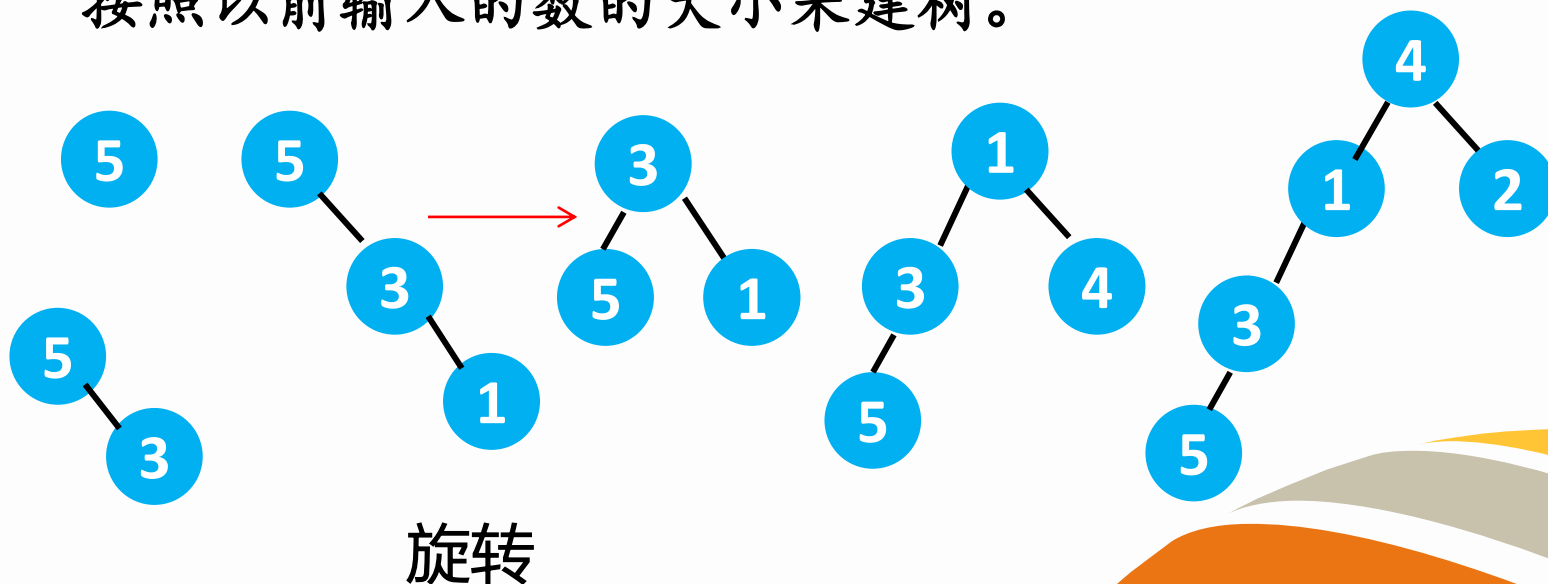
三：求已知序列中数 x 是区间 $[a, b]$ 第几大的数

求解方法和前面一样，找到前驱和后继，把区间 $[a, b]$ 变为一棵子树，再在这颗子树上查找数 x 。

序列问题

通过前面我们对Splay树的学习，我们知道二叉搜索树的中序遍历就是把序列按照从小到大的顺序输出。但是如果我们要保证原有序列的顺序性应该如何建树呢？比如输入顺序为5 3 1 4 2。

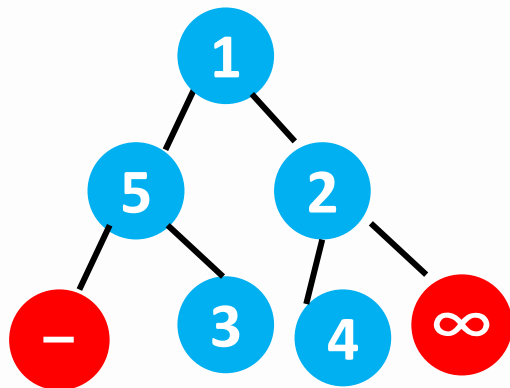
此时我们就应该按照该数的输入顺序来建树，不再是按照以前输入的数的大小来建树。



建树

我们会发现对于已知序列，因为序列编号肯定是从小到大，所以这样构建的平衡树雏形效率不太高，虽然后期每次查询都会变换平衡树的形态，所以我们可以优化一下，如果对于开始就已知序列。我们可以直接构建一棵完全二叉搜索树。

对于二叉搜索树来说，树的中序遍历就是原序列的顺序，所以我们每次用当前区间的中间节点作为二叉搜索树的根就可以了。同时因为要查找前驱和后继，所以我们最好在两端各添加一个虚点 $(-\infty, \infty)$





学习博客

Splay旋转细讲:

https://blog.csdn.net/qq_30974369/article/details/77587168

Splay基本操作:

<https://blog.csdn.net/luckcircle/article/details/73277977>

<https://www.cnblogs.com/five20/p/8313385.html>

<https://blog.csdn.net/octopusflying/article/details/51888826>





练习题

洛谷 P4146 序列终结者

洛谷 P2042 维护数列

洛谷 P1110 报表统计

